



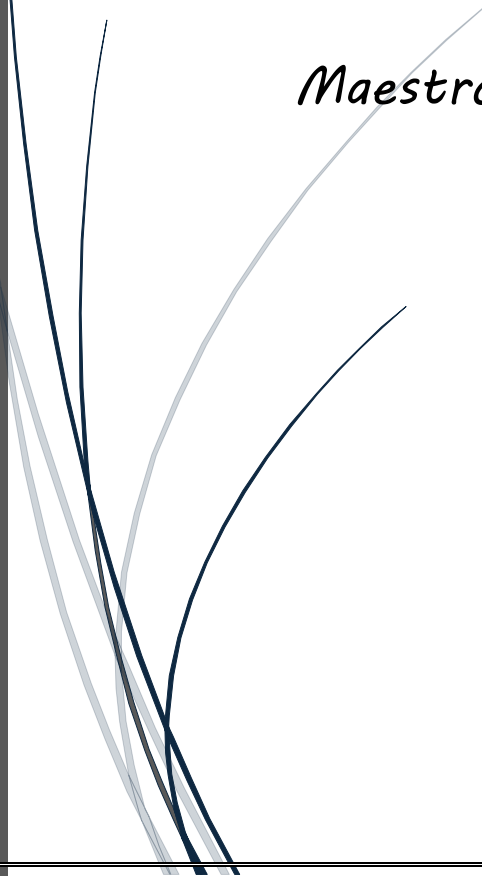
Universidad Tecnológica De Nezahualcóyotl

Documentación Ecommerce

Alumno(a): Solis Garcia Fany Sarai

Materia: Aplicaciones Web Orientada a Objetos

Maestro: Del Prado López Jovan



1 ¿Qué hace esta pagina?

The Look Lab es una **tienda de moda online** con 12 productos de la colección invernal 2026. Desde la página principal el usuario puede ver el catálogo, hacer clic en cualquier producto para ver sus detalles, elegir talla y cantidad, y agregarlo a un carrito lateral. Al hacer checkout, el sistema le pide su correo y método de pago, le envía un **código de verificación de 4 dígitos** por email usando EmailJS, y al confirmarlo genera un ticket de compra y envía un correo de confirmación. Los pedidos se guardan en el **localStorage** del navegador para que el usuario pueda revisarlos desde 'Mis Pedidos'.

2 Estructura HTML — index.html

Elemento / ID	Qué hace
nav	Barra de navegación pegajosa con el botón 'Mis Pedidos', el logo centrado y el ícono del carrito con contador
header.winter-header	Encabezado visual con el título 'COLECCIÓN INVIERNO' y el año de la edición
main#gridProductos	Cuadrícula donde JavaScript inyecta las tarjetas de los 12 productos al cargar la página
aside#sideCart	Carrito lateral que se desliza desde la derecha cuando el usuario agrega un producto
div#modalPago	Modal de pago con tres pasos: datos, verificación y ticket
div#paso1	Primer paso del modal: campo de correo y selección de método de pago
div#paso2	Segundo paso: campo para ingresar el código de 4 dígitos enviado por email
div#pasoTicket	Tercer paso: muestra el ticket de compra con folio, items y fecha de entrega
div#seccionHistorial	Panel de historial que se abre al pulsar 'Mis Pedidos'; lee los pedidos del localStorage
script (EmailJS CDN)	Carga la librería de EmailJS desde CDN para poder enviar correos desde el frontend

⚠ Todos los bloques de contenido dinámico (carrito, modal, historial) están en el HTML pero ocultos. JavaScript los llena y los muestra según las acciones del usuario.

3 Estilos CSS — style.css

nav — Barra de navegación con logo centrado

```
nav { display: flex; justify-content: space-between; padding: 25px 50px; position: sticky; top: 0; z-index: 1000; } .logo-center { position: absolute; left: 50%; transform: translateX(-50%); }
```

La nav usa Flexbox para alinear los tres elementos: botón izquierda, logo y carrito derecha. El logo se centra con position:absolute y translateX(-50%) en vez de estar en el flujo normal de Flexbox, lo que garantiza que siempre quede exactamente en el medio sin importar el tamaño de los otros elementos.

.grid-container — Cuadrícula de productos

```
.grid-container { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 40px; padding: 40px; }
```

CSS Grid con auto-fill crea tantas columnas como quepan con un mínimo de 280px. En pantallas grandes salen 4 columnas, en tablet 2, en móvil 1. Sin media queries.

.img-wrapper — Contenedor de imagen con proporción fija

```
.img-wrapper { aspect-ratio: 3 / 4; overflow: hidden; } .card img { width: 100%; height: 100%; object-fit: cover; transition: 0.8s; } .card:hover img { transform: scale(1.05); }
```

aspect-ratio:3/4 mantiene la proporción retrato de las fotos sin importar el ancho de la columna. object-fit:cover recorta la imagen para que llene el espacio sin deformarse. El hover hace un zoom suave de 5% en 0.8 segundos.

.cart-sidebar — Carrito deslizante

```
.cart-sidebar { position: fixed; right: -400px; width: 350px; height: 100%; transition: 0.5s; z-index: 2000; } .cart-sidebar.active { right: 0; }
```

El carrito empieza fuera de pantalla (right:-400px). Cuando JavaScript agrega la clase 'active', la propiedad right pasa a 0 y la transición de 0.5s crea el deslizamiento. El z-index 2000 lo pone por encima del contenido pero por debajo del modal de pago (z-index 3000).

.modal — Pantalla de pago

```
.modal { position: fixed; width: 100%; height: 100%; background: rgba(255,255,255,0.96); display: none; justify-content: center; align-items: center; z-index: 3000; }
```

Cubre toda la pantalla con un blanco casi opaco (0.96) que da un efecto elegante sin perder completamente el fondo. Solo aparece cuando display cambia de 'none' a 'flex' desde JavaScript.

.historial-overlay — Panel de historial

```
.historial-overlay { position: fixed; width: 100%; height: 100%; background: #fff; z-index: 4000; overflow-y: auto; }
```

El historial ocupa toda la pantalla con z-index 4000, el más alto de todos los elementos, por encima del modal y del carrito. overflow-y:auto permite hacer scroll si hay muchos pedidos guardados.

4 Catálogo de Productos

Los 12 productos viven en un array llamado **productos** al inicio del script.js. Cada objeto tiene estas propiedades:

Propiedad	Tipo	Para qué sirve
id	Número	Identificador único del producto. Se usa para buscarlo en el array al abrir el detalle o al agregar al carrito
tipo	Texto	Categoría: 'ropa', 'calzado' o 'accesorio'. Define si se muestra la tabla de tallas o no
nombre	Texto	Nombre del producto que aparece en la tarjeta y en la página de detalle
precio	Número	Precio unitario en MXN. El subtotal se calcula multiplicando precio x cantidad
imgFrente	Ruta	Ruta a la foto de frente del producto en la carpeta assets/
imgAtras	Ruta	Ruta a la foto de atrás. Se usa como miniatura en el detalle
tallas	Array	Lista de tallas disponibles. Se convierte en botones de selección en el detalle
desc	Texto	Descripción larga del producto que aparece en la página de detalle

Los 12 productos de la colección

#	Nombre	Tipo	Precio	Tallas
1	Abrigo Teddy Bear	Ropa	\$1,450	S, M, L
2	Impermeable Crystal	Ropa	\$890	Única
3	Sudadera Polar Paw	Ropa	\$750	S, M, L
4	Botas Lluvia Pastel	Calzado	\$980	23, 24, 25
5	Bufanda Infinita	Accesorio	\$350	Uni
6	Chaqueta Puffer Pro	Ropa	\$1,800	M, L
7	Gorro Orejitas	Accesorio	\$290	Uni
8	Trench Coat Classic	Ropa	\$1,650	S, M
9	Leggings Térmicos	Ropa	\$420	CH-M, G-XG
10	Paraguas Pocket Lab	Accesorio	\$320	Mini
11	Poncho colorado	Ropa	\$520	CH-M, G-XG
12	Sudadera Gatos en Enredo	Ropa	\$520	CH-M, G-XG

```
// Así se renderizan las tarjetas al cargar la página productos.forEach(p => {    grid.innerHTML
+= `        <article class='card' onclick='abrirPaginaDetalle(${p.id})'>        <div class='img-
wrapper'><img src='${p.imgFrente}'></div>        <p class='p-name'>${p.nombre}</p>        <p
class='p-price'>${p.precio} MXN</p>        </article>`; });
```

⚠ Cada tarjeta tiene un onclick que llama a `abrirPaginaDetalle(id)`. El id viaja como argumento para que la función sepa cuál producto abrir.

5 Página de Detalle del Producto

Al hacer clic en una tarjeta se ejecuta **abrirPaginaDetalle(id)**, que abre una nueva pestaña del navegador con **window.open()** y construye el HTML completo del detalle usando **v.document.write()**. No hay una página separada guardada en el servidor; el contenido se genera en tiempo real con JavaScript.

Efecto de rotación 3D al pasar el cursor

La imagen del producto rota en el eje Y según la posición horizontal del cursor, dando la ilusión de que la prenda se puede ver en perspectiva:

```
const moveRotate = (posX) => {    const rect = box.getBoundingClientRect();    const rot =  
(((posX - rect.left) / box.clientWidth) - 0.5) * 80;    img.style.transform = 'rotateY(' + rot + 'deg)';  
};    box.addEventListener('mousemove', (e) => moveRotate(e.clientX));  
box.addEventListener('mouseleave', () => img.style.transform = 'rotateY(0deg)');
```

Cuando el cursor está en el extremo izquierdo la rotación es -40° , en el centro 0° y en el extremo derecho $+40^\circ$. Al salir el cursor regresa suavemente a 0 gracias a `transition:0.1s`.

Selección de talla y agregar al carrito

```
function sel(b, t) {    document.querySelectorAll('.t-btn').forEach(x =>  
x.classList.remove('active'));    b.classList.add('active');    st = t; // guarda la talla seleccionada  
}    function add() {    if (!st) return alert('Por favor selecciona una talla.');
```

```
window.opener.agregarAlCarrito(id, st, cantidad);    window.close(); }
```

`window.opener` hace referencia a la página principal que abrió esta ventana. Gracias a eso, **agregarAlCarrito()** se ejecuta en el contexto de la tienda principal aunque se llame desde la ventana del detalle. Después de agregar, la ventana del detalle se cierra sola.

⚠ La tabla de tallas con medidas en centímetros solo aparece en productos de tipo 'ropa' o 'calzado'. Los accesorios (tipo 'accesorio') no la muestran porque `tablaHTML` queda en cadena vacía.

6 Carrito Lateral

agregarAlCarrito(id, talla, cantidad)

Busca el producto en el array por su id, crea un objeto con todos sus datos más la talla elegida y el subtotal, y lo empuja al array carrito. Luego llama a actualizarUI() y abre el carrito automáticamente si estaba cerrado.

```
function agregarAlCarrito(id, talla, cantidad) {  const pBase = productos.find(x => x.id === id);
carrito.push({      ...pBase,      tallaElegida: talla,      cantidad: cantidad,      subtotal:
pBase.precio      *      cantidad      });      actualizarUI();      if
(!document.getElementById('sideCart').classList.contains('active'))      toggleCarrito(); }
```

actualizarUI()

Reconstruye todo el HTML del carrito lateral desde cero cada vez que hay un cambio. Calcula el total sumando los subtotales con reduce() y lo muestra en la parte inferior junto al botón de Checkout.

```
const total = carrito.reduce((s, i) => s + i.subtotal, 0); // Cada item en el carrito:
`<div>${i.nombre} (x${i.cantidad}) - Talla: ${i.tallaElegida}<br>${i.subtotal}</div>`
```

⚠ El carrito no persiste entre sesiones. Si el usuario cierra o recarga la página, el array carrito se reinicia y los productos desaparecen. Solo los pedidos finalizados se guardan en localStorage.

7 Flujo de Pago y Verificación

7.1 Paso 1 — Correo y método de pago

Al hacer clic en Checkout, `abrirVerificacion()` inyecta el formulario del paso 1 dentro del modal y lo hace visible. El usuario ingresa su correo y elige entre cuatro métodos de pago: Tarjeta, PayPal, OXXO o SPEI. El botón PAGAR AHORA no aparece hasta que se elige un método.

```
function seleccionarMetodo(el, met) { document.querySelectorAll('.pay-method').forEach(m  
=> m.classList.remove('selected')); el.classList.add('selected'); metodoSeleccionado = met;  
}
```

7.2 EmailJS — Envío del código de verificación

EmailJS es una API que permite enviar correos directamente desde el frontend sin necesidad de un servidor backend. Se inicializa con una clave pública al cargar el script, y para enviar un correo solo se llama a `emailjs.send()` con el ID del servicio y la plantilla.

```
// Inicialización al cargar emailjs.init('WDGWcCoDSfaytwhKL'); // Envío del código de  
verificación codGen = Math.floor(1000 + Math.random() * 9000); // código de 4 dígitos  
emailjs.send('service_x64il4c', 'template_lpaue55', { email: emailDelUsuario,  
codigo_verificacion: codGen, metodo: metodoSeleccionado }).then(() => { //  
muestra el paso 2 });
```

`Math.random() * 9000 + 1000` genera un número entre 1000 y 9999, siempre de 4 dígitos. Este código se guarda en la variable global `codGen` para compararlo cuando el usuario lo ingrese.

7.3 Paso 2 — Confirmar el código

```
function finalizarCompra() { if (document.getElementById('inputCodigo').value == codGen) {  
// código correcto: genera ticket y envía correo generarTicket(orden); } else {  
alert('Código incorrecto.');
```

La comparación usa `==` (no `===`) para que funcione aunque `inputCodigo.value` sea string y `codGen` sea número. Si el código no coincide simplemente muestra una alerta.

7.4 Ticket y correo de confirmación

Si el código es correcto, se calculan el folio (número aleatorio), la fecha de entrega (hoy + 7 días) y se manda un segundo correo con los detalles del pedido. Luego `generarTicket()` muestra el paso 3 con el comprobante visual y guarda el pedido en `localStorage`.

```
const entrega = new Date(); entrega.setDate(entrega.getDate() + 7);  
emailjs.send('service_x64il4c', 'template_f8zqwwv', { email, order_id, total, items, metodo,  
fecha, fecha_entrega }); // Guardar en historial let hist =  
JSON.parse(localStorage.getItem('L_Hist')) || []; hist.push(orden); localStorage.setItem('L_Hist',  
JSON.stringify(hist));
```

⚠ Se usan dos plantillas distintas de EmailJS: `template_lpaue55` para el código de verificación y `template_f8zqwwv` para el ticket de compra. Cada plantilla tiene sus propias variables configuradas en el panel de EmailJS.

8 Historial de Pedidos

Al pulsar 'Mis Pedidos' en la barra de navegación se abre el panel de historial. `toggleHistorial()` lee el array guardado en `localStorage` bajo la clave 'L_Hist', lo invierte con `reverse()` para mostrar el más reciente primero, y genera el HTML de cada pedido.

```
function toggleHistorial() {    const ped = JSON.parse(localStorage.getItem('L_Hist')) || [];
document.getElementById('contenedorPedidos').innerHTML =    ped.reverse().map(p => `
<div>                <b>PEDIDO #${p.id}</b> - ${p.fecha}<br>                Items: ${p.items}<br>
Total: <b>${p.total}</b> MXN</b>                </div>` ).join("") || 'Sin pedidos.'; }
```

⚠ Los pedidos persisten en el navegador hasta que el usuario borre el `localStorage` manualmente o use modo incognito. Son específicos del dispositivo y navegador, no se sincronizan entre dispositivos.

9 Flujo Completo de una Compra

#	Actor	Qué pasa
1	Navegador	Carga index.html, importa EmailJS desde CDN e inicializa con la clave pública
2	JavaScript	forEach recorre los 12 productos y renderiza sus tarjetas en el grid
3	Usuario	Hace clic en una tarjeta del catálogo
4	abrirPaginaDetalle()	Abre una nueva pestaña y escribe el HTML del detalle con document.write()
5	Usuario	Pasa el cursor sobre la imagen: se activa la rotación 3D en el eje Y
6	Usuario	Selecciona una talla y una cantidad, pulsa 'AÑADIR A LA BOLSA'
7	agregarAlCarrito()	Agrega el producto al array carrito y actualiza el contador de la nav
8	toggleCarrito()	El carrito lateral se desliza desde la derecha automáticamente
9	Usuario	Pulsa CHECKOUT en el carrito
10	abrirVerificacion()	El modal de pago aparece con el formulario del paso 1
11	Usuario	Escribe su correo y elige un método de pago; pulsa SOLICITAR CÓDIGO
12	enviarCodigo()	Genera el código de 4 dígitos y lo envía al correo via EmailJS
13	EmailJS	Entrega el correo con el código al usuario en segundos
14	Usuario	Ingresa el código en el paso 2 y pulsa CONFIRMAR PEDIDO
15	finalizarCompra()	Valida el código; si es correcto calcula fecha de entrega y folio
16	EmailJS	Envía el ticket de compra completo al correo del usuario
17	generarTicket()	Muestra el comprobante en el paso 3 y guarda el pedido en localStorage
18	Usuario	Pulsa 'VOLVER A LA TIENDA': location.reload() reinicia la página